

DOCUMENTAÇÃO PARA SEMINÁRIO

GenRoute GA - Algoritmo Genético aplicado à Otimização de Rotas

Uma abordagem computacional para o Problema do Caixeiro Viajante

Discente: Iury Figueiredo Lopes

Docente: Danilo Dias da Silva

Tema

Algoritmo Genético aplicado à otimização combinatória de rotas.

Problema aplicado

Problema do Caixeiro Viajante (TSP), com possibilidade de extensão futura para roteirização de veículos (VRP).

Tecnologias utilizadas

Java, Maven, JavaFX, VS Code e Git/GitHub.

Observação

Esta documentação foi organizada para servir tanto como material entregue ao professor quanto como roteiro de apoio durante a apresentação. O projeto citado neste documento foi implementado no repositório GenRoute GA, com problemas em arquivos CSV, execução por Maven e visualização gráfica animada em JavaFX.

Delimitação do foco do trabalho

O foco principal deste projeto não é apenas resolver uma rota específica. O foco é demonstrar como um Algoritmo Genético pode ser aplicado, configurado, executado e analisado em um problema de otimização combinatória.

O Problema do Caixeiro Viajante foi escolhido como cenário de teste porque permite visualizar claramente todas as etapas do Algoritmo Genético: criação da população inicial, avaliação por função de aptidão, seleção, cruzamento, mutação, elitismo e acompanhamento da evolução por gerações. Assim, o TSP funciona como o problema aplicado, enquanto o Algoritmo Genético é o centro da análise computacional.

1. Introdução e contextualização

Em muitas situações do cotidiano, tomar uma boa decisão significa escolher a melhor ordem para executar várias tarefas. Uma empresa de entregas precisa decidir em qual sequência seus clientes serão atendidos. Um técnico de manutenção precisa visitar diferentes endereços no mesmo dia. Um serviço de coleta precisa passar por vários pontos da cidade sem desperdiçar tempo, combustível e equipe. Mesmo quando essas decisões parecem simples, elas escondem um problema matemático importante: encontrar uma rota eficiente entre diversos pontos.

Esse tipo de decisão está diretamente ligado à otimização combinatória, área que estuda problemas em que é necessário escolher a melhor alternativa dentro de um conjunto muito grande de combinações possíveis. Em problemas de rota, cada combinação representa uma ordem diferente de visita. Quando existem poucos pontos, ainda é possível comparar algumas possibilidades manualmente. Porém, conforme a quantidade de pontos aumenta, o número de rotas possíveis cresce rapidamente, tornando inviável testar todas as combinações uma por uma.

Nesse contexto, a computação passa a ter um papel essencial. Em vez de buscar a solução por tentativa manual ou por força bruta, pode-se utilizar técnicas de busca e otimização capazes de encontrar boas soluções em tempo viável. O Algoritmo Genético é uma dessas técnicas. Ele se inspira no processo de evolução natural e trabalha com uma população de soluções candidatas, selecionando, combinando e modificando rotas ao longo de várias gerações.

O objetivo deste seminário é mostrar como a teoria de Algoritmos Genéticos pode ser aplicada em um problema concreto, visual e próximo de situações reais: a otimização de rotas. Assim, o trabalho relaciona modelagem matemática, implementação em Java e visualização gráfica dos resultados, permitindo observar a evolução de uma rota inicial aleatória até uma rota melhor encontrada pelo algoritmo.

2. Problema trabalhado e importância prática

O problema central deste projeto é o Problema do Caixeiro Viajante, conhecido internacionalmente como Traveling Salesman Problem (TSP). Nesse problema, existe um conjunto de cidades, clientes ou pontos de entrega. O objetivo é encontrar a menor rota possível que visite todos os pontos exatamente uma vez e retorne ao ponto inicial.

De forma aplicada, imagine uma empresa que precisa visitar vários clientes durante uma rota de entrega. A pergunta principal é: em qual ordem esses clientes devem ser visitados para reduzir a distância total percorrida? Essa decisão tem impacto direto no custo da operação, no tempo de atendimento, no consumo de combustível, na organização da equipe e na qualidade do serviço prestado.

A importância desse teste para a vida real está no fato de que problemas de rota aparecem em várias atividades do dia a dia. Aplicativos de entrega, empresas de transporte, serviços de coleta, equipes de manutenção, visitas comerciais, atendimento técnico em campo e planejamento urbano dependem de decisões semelhantes. Mesmo quando o sistema real é mais complexo que o TSP, esse problema serve como base para compreender como a roteirização pode ser modelada e otimizada.

Na prática, sistemas de logística não podem depender apenas de escolhas manuais. Uma rota mal planejada pode gerar atrasos, aumentar gastos, reduzir produtividade e dificultar a execução de projetos. Por outro lado, uma rota otimizada ajuda a usar melhor os recursos disponíveis. Isso é importante principalmente em projetos que envolvem deslocamento, prazos, múltiplos pontos de atendimento e necessidade de acompanhar resultados por indicadores.

Por isso, o TSP é um bom problema para este seminário. Ele é simples de explicar, mas matematicamente relevante. Também permite demonstrar visualmente a diferença entre uma rota inicial e uma rota otimizada. Dessa forma, o teste mostra como uma técnica computacional pode apoiar decisões reais e melhorar a execução de projetos que dependem de planejamento de rotas.

3. Justificativa da escolha

- Relação direta com Algoritmo Genético: cada solução candidata pode ser representada como uma rota. A população é formada por várias rotas, e as melhores rotas são combinadas e modificadas ao longo das gerações.
- Demonstração completa dos operadores genéticos: o projeto permite mostrar, no mesmo exemplo, população, indivíduo, função de aptidão, seleção, cruzamento, mutação e elitismo.
- Força em otimização combinatória: o número de rotas possíveis cresce rapidamente conforme a quantidade de pontos aumenta. Por isso, a busca exaustiva se torna inviável para instâncias maiores.
- Comparação entre busca exaustiva e busca heurística: o projeto deixa claro que o Algoritmo Genético não tenta testar todas as rotas, mas busca boas soluções por evolução de uma população.
- Facilidade visual para a turma: a rota inicial, a melhor rota encontrada e o gráfico de evolução podem ser exibidos na tela, facilitando a compreensão do processo.
- Aplicação real: o mesmo tipo de raciocínio aparece em entregas, transporte, coleta de materiais, roteirização de técnicos e distribuição de produtos.
- Ligação com projetos reais: a abordagem mostra como um problema pode sair da modelagem matemática, passar pela implementação computacional e chegar a uma visualização útil para tomada de decisão.

4. Modelagem matemática do problema

Considere um conjunto de pontos ou cidades:

$$V = \{1, 2, 3, \dots, n\}$$

Cada cidade i possui coordenadas (x_i, y_i) . A distância entre duas cidades i e j pode ser calculada pela distância euclidiana:

$$d_{ij} = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}$$

A rota é uma permutação das cidades, isto é, uma ordenação possível dos pontos visitados. O objetivo é minimizar a soma das distâncias entre cidades consecutivas, incluindo o retorno ao ponto inicial:

$$\min Z = \text{soma das distâncias entre pontos consecutivos} + \text{retorno ao ponto inicial}$$

Restrições principais:

- cada cidade deve aparecer exatamente uma vez na rota;
- todas as cidades devem ser visitadas;
- a última cidade deve se conectar novamente ao ponto inicial.

5. Como o Algoritmo Genético é aplicado

No projeto, cada indivíduo da população representa uma rota completa. O ponto 0 é mantido como ponto inicial para facilitar a leitura durante a apresentação.

Exemplo de indivíduo:

$$[0, 3, 5, 1, 2, 4]$$

Essa rota representa:

Ponto 0 -> Ponto 3 -> Ponto 5 -> Ponto 1 -> Ponto 2 -> Ponto 4 -> retorno ao Ponto 0

Componentes utilizados:

- Indivíduo ou cromossomo: uma rota completa.
- População inicial: conjunto de rotas aleatórias.
- Função de aptidão: avaliação baseada na distância total da rota. Quanto menor a distância, melhor a solução.
- Seleção: seleção por torneio, dando maior chance às rotas com menor distância.
- Cruzamento: Ordered Crossover, que combina trechos de duas rotas sem repetir cidades.
- Mutação: troca de duas cidades de posição, criando variação nas soluções.
- Elitismo: preservação das melhores rotas para evitar perder a melhor solução já encontrada.

O ciclo do Algoritmo Genético no GenRoute GA ocorre da seguinte forma:

1. O sistema carrega uma lista de cidades a partir de um arquivo CSV.
2. O algoritmo cria uma população inicial com várias rotas aleatórias.
3. Cada rota é avaliada pela distância total percorrida.
4. As rotas com menor distância têm maior chance de serem selecionadas.
5. Duas rotas selecionadas são combinadas pelo cruzamento ordenado.
6. Algumas rotas sofrem mutação, trocando duas cidades de posição.
7. As melhores rotas são preservadas pelo elitismo.
8. O processo se repete por várias gerações.
9. Ao final, o sistema apresenta a melhor rota encontrada e o histórico de evolução.

Essa sequência é o ponto central do projeto. A rota é o exemplo aplicado, mas a contribuição principal está em demonstrar como o Algoritmo Genético transforma uma população inicial de soluções aleatórias em soluções progressivamente melhores.

5.1 Por que o Algoritmo Genético é adequado para este projeto

O Algoritmo Genético é adequado porque trabalha bem com problemas em que existem muitas combinações possíveis e em que testar todas as alternativas seria inviável. No caso das rotas, cada ordem diferente de visita representa uma solução. Como o número de ordens cresce muito rapidamente, o algoritmo utiliza uma estratégia evolutiva para procurar boas soluções.

Além disso, o problema permite explicar os principais conceitos do AG de forma concreta:

- o cromossomo é a sequência de cidades;
- o gene é cada cidade dentro da rota;
- a população é o conjunto de rotas;
- o fitness é calculado pela distância total;
- a seleção favorece rotas menores;
- o cruzamento combina trechos de rotas promissoras;
- a mutação cria novas variações;
- o elitismo impede a perda da melhor rota encontrada.

Portanto, o projeto foi organizado para que o funcionamento do Algoritmo Genético seja observável tanto no código quanto na interface gráfica.

6. Implementação computacional desenvolvida

O projeto foi implementado em Java com Maven para gerenciamento e JavaFX para a interface visual. A estrutura foi organizada para separar a lógica do algoritmo, os dados dos problemas e a visualização dos resultados.

Principais módulos:

- City: representa um ponto ou cidade com nome e coordenadas x e y.
- Route: representa uma rota, calcula sua distância total e mantém a ordem de visita.
- ProblemInstance: representa um problema carregado a partir de arquivo CSV.
- ProblemRepository: lê os problemas cadastrados na pasta data/problemas.
- GeneticParameters: armazena parâmetros como população, gerações, mutação, elitismo e torneio.
- GeneticAlgorithm: executa seleção, cruzamento, mutação, elitismo e evolução das gerações.
- GeneticResult: armazena a rota inicial, a melhor rota e os históricos de distância.
- RouteCanvas: desenha a rota inicial e a rota otimizada de forma visual.
- App: monta a interface JavaFX, os controles, o gráfico e a animação.

Estrutura dos problemas:

Os problemas ficam na pasta data/problemas. Cada arquivo CSV contém uma lista de pontos com nome e coordenadas. A primeira linha representa o ponto inicial da rota.

Exemplo:

nome,x,y Deposito,50,50 Cliente 01,18,72 Cliente 02,23,28

7. Resultados visuais esperados

A apresentação do projeto deve ser visual. A ideia é mostrar que o algoritmo não está apenas imprimindo números, mas melhorando uma solução que pode ser observada graficamente.

O sistema apresenta:

- rota inicial gerada aleatoriamente;
- melhor rota encontrada pelo Algoritmo Genético;
- distância inicial;
- distância final;
- percentual de melhoria;
- gráfico animado da evolução da distância ao longo das gerações;
- comparação visual entre rota inicial e rota otimizada.

Exemplo de leitura do resultado:

Se a distância inicial for 850,42 e a distância final for 374,18, a melhoria será aproximadamente 56%. Isso mostra que a solução final percorre uma distância bem menor que a solução gerada aleatoriamente no início.

8. Cenários de teste para o seminário

Os testes foram pensados para facilitar a explicação em sala e mostrar a evolução do algoritmo em diferentes tamanhos de problema.

- Teste pequeno, com 10 pontos: usado para explicar a lógica sem poluir a tela.
- Teste médio, com 20 pontos: usado para mostrar melhor o comportamento evolutivo.
- Teste maior, com 30 pontos: usado para evidenciar a dificuldade combinatória e a utilidade do Algoritmo Genético.
- Extensão futura, com múltiplos veículos: aproximação do Problema de Roteirização de Veículos (VRP), mais próximo de cenários logísticos reais.

9. Organização dos dados pesquisados de referência

As fontes foram organizadas em grupos para facilitar o uso durante a escrita, a fala e a justificativa técnica do projeto.

As referências mais importantes para o foco do trabalho são as referências sobre Algoritmos Genéticos. Elas sustentam a explicação de população, indivíduo, função de aptidão, seleção, cruzamento, mutação, elitismo e processo evolutivo.

- Base teórica de AG: conceitos gerais como população, indivíduos, operadores genéticos e processo evolutivo.
- Base clássica de AG: Holland e Goldberg, por serem referências fundamentais sobre algoritmos genéticos, adaptação, busca e otimização.
- Base introdutória de AG: explicações em linguagem acessível para apoiar a apresentação.
- Problema matemático de teste: definição do TSP e instâncias clássicas de teste. Essas referências justificam o problema escolhido, mas não substituem o foco no Algoritmo Genético.
- Aplicação real: roteamento em entregas, técnicos, transporte e logística. Essas referências ajudam a contextualizar por que o teste é relevante.
- Tecnologia visual: JavaFX como plataforma para interface desktop e gráficos.

Assim, as referências de TSP, logística e JavaFX aparecem como apoio. O eixo principal da fundamentação continua sendo o uso do Algoritmo Genético como técnica de busca e otimização.

10. Referências

ICMC-USP. Algoritmos Genéticos. Disponível em: <https://sites.icmc.usp.br/andre/research/genetic/>. Acesso em: 23 jun. 2026.

RAPADURA TECH. Introdução aos Algoritmos Genéticos. Medium. Disponível em: <https://medium.com/rapaduratech/introdu%C3%A7%C3%A3o-aos-algoritmos-gen%C3%A9ticos-405825c1e281>. Acesso em: 23 jun. 2026.

REINELT, Gerhard. TSPLIB - A Traveling Salesman Problem Library. ORSA Journal on Computing, v. 3, n. 4, p. 376-384, 1991. DOI: 10.1287/ijoc.3.4.376.

TSPLIB. TSPLIB95. University of Heidelberg. Disponível em: <https://comopt.ifi.uni-heidelberg.de/software/TSPLIB95/index.html>. Acesso em: 23 jun. 2026.

GOOGLE DEVELOPERS. Vehicle Routing | OR-Tools. Disponível em: <https://developers.google.com/optimization/routing>. Acesso em: 23 jun. 2026.

GOOGLE DEVELOPERS. Vehicle Routing Problem | OR-Tools. Disponível em: <https://developers.google.com/optimization/routing/vrp>. Acesso em: 23 jun. 2026.

OPENJFX. JavaFX. Disponível em: <https://openjfx.io/>. Acesso em: 23 jun. 2026.

OPENJFX. Getting Started with JavaFX. Disponível em: <https://openjfx.io/openjfx-docs/>. Acesso em: 23 jun. 2026.

HOLLAND, John H. Adaptation in Natural and Artificial Systems. Ann Arbor: University of Michigan Press, 1975.

GOLDBERG, David E. Genetic Algorithms in Search, Optimization, and Machine Learning. Reading: Addison-Wesley, 1989.

11. Roteiro curto de fala

1. Comece deixando claro que o foco do seminário é o Algoritmo Genético como técnica de otimização.
2. Explique que o TSP foi escolhido como problema de teste porque permite visualizar bem o funcionamento do AG.
3. Antes de apresentar o problema, contextualize decisões reais que envolvem escolher a melhor ordem de execução de tarefas.
4. Apresente o TSP como uma versão matemática desse tipo de decisão.
5. Explique que o desafio está no crescimento rápido do número de rotas possíveis.
6. Mostre que cada indivíduo do Algoritmo Genético representa uma rota.
7. Explique que a qualidade da rota é medida pela distância total.
8. Mostre seleção, cruzamento, mutação e elitismo no código.
9. Mostre a execução visual: rota inicial, rota otimizada e gráfico de evolução.
10. Conclua destacando que o AG ajuda a encontrar boas soluções quando testar todas as possibilidades não é viável.

12. Conclusão

O projeto GenRoute GA propõe uma aplicação prática de Algoritmo Genético em um problema clássico de otimização combinatória. Ao trabalhar com o Problema do Caixeiro Viajante, é possível relacionar matemática, programação e situações reais de logística sem perder o foco principal: demonstrar o funcionamento do Algoritmo Genético.

A principal contribuição do projeto é mostrar como os operadores genéticos atuam sobre uma população de soluções. A visualização gráfica ajuda a tornar esse processo mais claro, pois permite comparar uma rota inicial aleatória com a melhor rota encontrada após várias gerações e acompanhar a evolução da aptidão no gráfico.

Assim, o trabalho demonstra como a Matemática Computacional pode apoiar a execução de projetos reais, principalmente quando há muitas combinações possíveis e a busca manual ou exaustiva deixa de ser viável. O TSP é o cenário aplicado; o Algoritmo Genético é o método central estudado, implementado e apresentado.

13. Checklist de apoio durante a apresentação

- Início: apresentar o contexto antes do problema, usando exemplos reais de rotas.
- Foco: deixar claro que o TSP é o cenário de teste e que o centro do projeto é o Algoritmo Genético.
- Problema: explicar o TSP como decisão sobre a melhor ordem de visita.
- Modelagem: mostrar pontos, distâncias e função objetivo.
- Código: apresentar principalmente GeneticAlgorithm, GeneticParameters, GeneticResult, Route e City.
- Execução: mostrar rota inicial, rota final e gráfico animado.
- Conclusão: destacar que a melhoria percentual é consequência do processo evolutivo do AG.

Perguntas prováveis e respostas curtas:

- O algoritmo sempre acha a melhor rota? Não necessariamente. Ele busca uma solução boa ou próxima do ótimo, especialmente quando testar tudo seria inviável.
- Por que a mutação é importante? Porque ela cria variação e ajuda o algoritmo a escapar de soluções ruins ou repetidas.
- Por que usar Java? Porque Java permite organizar bem o projeto por classes e usar JavaFX para visualização gráfica.
- Qual é a aplicação real? Roteirização de entregas, visitas técnicas, transporte, coleta e logística.
- Por que esse teste é importante para projetos? Porque mostra como medir, comparar e melhorar uma solução antes de aplicar a lógica em cenários maiores.